INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

UNIDAD DE APRENDIZAJE

Ingeniería de Pruebas

PRÁCTICA

Pruebas de Penetración

SECUENCIA Y PERIODO

6NM61 2026-2

BOLETA

2021601769

ALUMNOS

- Albarrán Soto Alexis Kevin
- Camacho Colin Camila Danae
- Ferrusca Pineda Mildred Arely
- Gonzalez Calzada Maximiliano
- León Merino Ángel Mauricio
- Ramírez Lescas Eliu
- Sosa Montoya Melanie Rubí

PROFESORA

González Arroyo Lilia

FECHA

Abril de 2026

1. ¿Qué son las Pruebas de Penetración?

Las **pruebas de penetración** (o *pen testing*, del inglés *penetration testing*) son un conjunto de técnicas de evaluación de seguridad en las que un profesional autorizado simula ataques reales contra un sistema, aplicación o infraestructura de TI, con el objetivo de identificar vulnerabilidades antes de que puedan ser explotadas por actores maliciosos. A diferencia de los análisis de vulnerabilidades automatizados, las pruebas de penetración implican un proceso activo y creativo de explotación controlada.

El profesional encargado de llevar a cabo estas pruebas debe contar con conocimiento de la arquitectura del sistema objetivo, las tecnologías involucradas y los vectores de ataque más comunes. El resultado del proceso es un reporte que documenta las vulnerabilidades descubiertas, su severidad y las recomendaciones para mitigarlas.

1.1. Tipos de Pruebas de Penetración

Dependiendo del nivel de información con que cuenta el evaluador, las pruebas de penetración se clasifican en tres categorías principales:

Tipo	Acceso al código / documentación	Ataque simulado	Aspectos evaluados
Caja Blanca	Acceso completo al código fuente y documentación del sistema	Empleado interno con privilegios (insider threat)	Fallas lógicas, caminos mal estructurados, flujo de entradas, ciclos y condiciones, hoyos de seguridad interna, funciones individuales
Caja Negra	Sin acceso al código ni documentación; solo versión pública de la aplicación	Ataque externo (hacker) — intercepción de tráfico e inyección de contenido malicioso	XSS, inyección SQL, inyección de comandos, inyección de links, intercepción de tráfico
Caja Gris	Acceso parcial al código y documentación, usados como guía	Combinación de ambos escenarios anteriores	Análisis con ventaja informacional limitada; combina técnicas de caja blanca y negra

Tabla 1: Clasificación de las pruebas de penetración según el nivel de acceso al sistema

1.2. Ethical Hacking

El *ethical hacking* (o **hacking ético**) es la práctica de explotar sistemas informáticos de forma deliberada y autorizada para identificar vulnerabilidades de seguridad, antes de que un actor malicioso pueda aprovecharlas. A diferencia del hacking malicioso, el hacking ético se realiza siempre con el conocimiento y el consentimiento explícito del propietario del sistema objetivo.

1.2.1. Principios fundamentales

- **Autorización explícita:** El evaluador trabaja bajo un contrato o acuerdo que delimita claramente el alcance de las pruebas. Actuar fuera de ese alcance constituye un delito.
- **Alcance delimitado:** Solo se evalúan los sistemas, redes o aplicaciones específicamente listados en el acuerdo. Cualquier sistema fuera del alcance es intocable.
- **Reporte responsable:** Los hallazgos se documentan en un reporte detallado que se entrega confidencialmente al cliente, con severidad, evidencia y recomendaciones.
- **No daño intencional:** El objetivo es identificar vulnerabilidades, no explotar daños permanentes ni comprometer datos reales.

1.2.2. Relación con las pruebas de penetración

El hacking ético abarca las pruebas de penetración como su técnica central, pero también incluye auditorías de configuración, análisis de riesgo, ingeniería social controlada (phishing simulado) y revisión de arquitecturas de seguridad. Toda prueba de penetración es hacking ético, pero no todo hacking ético es necesariamente una prueba de penetración.

1.2.3. Fases del proceso

1. **Reconocimiento (Information Gathering):** Recopilación pasiva y activa de información sobre el objetivo: dominios, IPs, tecnologías, empleados, servicios expuestos.

2. **Escaneo y enumeración:** Identificación de puertos abiertos, servicios activos, versiones de software y posibles puntos de entrada mediante herramientas como Nmap o Nessus.
3. **Explotación controlada:** Intento de aprovechar las vulnerabilidades descubiertas para ganar acceso no autorizado, siempre dentro del alcance acordado.
4. **Post-explotación y escalada de privilegios:** Evaluación del daño potencial en caso de que un atacante hubiera comprometido el sistema: movimiento lateral, acceso a datos, persistencia.
5. **Limpieza:** Eliminación de cualquier herramienta, backdoor o artefacto dejado durante las pruebas para restablecer el estado original del sistema.
6. **Reporte:** Documentación formal de vulnerabilidades, evidencia, severidad (CVSS), impacto potencial y recomendaciones de remediación.

1.3. QA y Pruebas de Seguridad

El aseguramiento de la calidad (*Quality Assurance*, QA) en el contexto de la seguridad del software tiene como objetivo garantizar que una aplicación no solo funcione correctamente, sino que también sea resistente a ataques y proteja adecuadamente la información de sus usuarios. A diferencia de las pruebas funcionales —que verifican que el sistema hace lo que se espera— las pruebas de seguridad verifican que el sistema no hace lo que **no** se espera cuando recibe entradas maliciosas o inesperadas.

1.3.1. Tipos de pruebas de seguridad en QA

Tipo	Qué analiza	Cuándo se aplica
SAST *(Static Application Security Testing)*	Código fuente, bytecode o binarios sin ejecutar la aplicación. Detecta inyecciones SQL, XSS, secretos embebidos, configuraciones inseguras.	Durante el desarrollo; integrado en el pipeline de CI/CD. Detecta problemas temprano (shift-left).
DAST *(Dynamic Application Security Testing)*	La aplicación en ejecución mediante simulación de ataques (fuzzing, escaneo de endpoints, manipulación de parámetros).	En entornos de staging o preproducción. Complementa al SAST detectando vulnerabilidades de runtime.
IAST *(Interactive Application Security Testing)*	Combina agentes instrumentados dentro de la app con pruebas dinámicas; mapea el flujo de datos durante la ejecución.	Durante pruebas funcionales o de integración. Mayor precisión, menor tasa de falsos positivos.
Fuzzing	Envío masivo de entradas aleatorias, malformadas o semi-válidas para detectar crashes, desbordamientos de buffer o comportamientos inesperados.	En etapas avanzadas de QA; especialmente útil para protocolos, parsers y APIs.
Pentesting (DAST manual)	Prueba de penetración completa conducida por un profesional humano, con mayor creatividad y contexto que las herramientas automáticas.	En releases mayores, auditorías periódicas o antes de producción.

Tabla 2: Tipos de pruebas de seguridad integradas en el proceso de QA

1.3.2. Integración con el ciclo de desarrollo (SDL / Shift-Left Security)

La práctica de *shift-left* promueve incorporar las pruebas de seguridad desde las fases más tempranas del ciclo de vida del software, en lugar de tratarlas como una etapa final. El marco de referencia del *Software Development Lifecycle* (SDL) de Microsoft y el *Secure Software Development Framework* (SSDF) del NIST formalizan este enfoque, exigiendo que cada fase del desarrollo —desde los requerimientos hasta el despliegue— contemple controles de seguridad específicos.

1.4. Vulnerabilidades Comunes por Arquitectura de Software

La arquitectura de un sistema define de forma determinante su superficie de ataque: cuántos puntos de entrada existen, qué tan aislados están los componentes entre sí, y cuál es el radio de daño (*blast radius*) en caso de una brecha exitosa. La siguiente tabla clasifica las arquitecturas más comunes de mayor a menor exposición general, considerando la combinación de superficie de ataque, facilidad de explotación y daño potencial.

Rango	Arquitectura	Vulnerabilidades principales	Blast Radius
🔴 1	**Monolítica**	SQL injection con acceso a toda la base de datos; RCE con control total del servidor; SSRF; auth bypass que expone toda la funcionalidad. Una sola brecha compromete el sistema completo.	Crítico
🔴 2	**MVC** *(Model-View-Controller)*	XSS en la capa View; SQL injection en la capa Model; CSRF en la capa Controller; IDOR (*Insecure Direct Object Reference*); validación de entrada inconsistente entre capas.	Alto
🟠 3	**N-capas** *(Layered / N-tier)*	Session hijacking entre capa de presentación y lógica de negocio; inyección de datos entre capas; escalada de privilegios si el aislamiento entre capas no es estricto; deserialización insegura.	Alto
🟠 4	**SOA / API-First** *(REST, GraphQL)*	Broken Object Level Authorization (BOLA/IDOR); JWT manipulation; ausencia de rate limiting; mass assignment; exposición excesiva de datos en respuestas; enumeración de endpoints.	Medio-Alto
🟡 5	**Microservicios**	Mayor superficie de ataque total (muchos endpoints); comunicación inter-servicio insegura (mTLS mal configurado); vulnerabilidades en el API gateway; imágenes de contenedor desactualizadas; secretos expuestos en variables de entorno.	Medio *(aislado por servicio)*
🟡 6	**Event-Driven** *(Pub-Sub, Message Queue)*	Inyección de mensajes maliciosos en colas; replay attacks; deserialización insegura del payload; falta de autenticación entre productor y consumidor. Difícil rastrear el origen del ataque.	Medio
🟢 7	**Serverless / FaaS**	Inyección por eventos de entrada (S3, API Gateway, etc.); dependencias con vulnerabilidades conocidas (supply chain); funciones con permisos excesivos (over-privileged IAM roles); cold start information leakage.	Bajo *(por función)*

Tabla 3: Arquitecturas de software ordenadas de mayor a menor exposición a vulnerabilidades de seguridad

Es importante notar que ninguna arquitectura es inherentemente «segura»: cada una introduce sus propios vectores de ataque. La arquitectura serverless, aunque tiene el menor radio de daño por unidad funcional, puede tener una cadena de dependencias compleja que amplíe significativamente la superficie de ataque total. Los microservicios, por su parte, reducen el blast radius per servicio, pero multiplican la superficie de ataque global y la complejidad del monitoreo.

2. Resumen: Introducción a las Pruebas de Penetración en Dispositivos iOS

Carlos Joaquín Brito Abundis — SG Buzz, Revista 42, sección Prácticas

2.1. Contexto y relevancia

Apple se ha consolidado como una de las marcas más valiosas del mundo, con alrededor de 700 millones de dispositivos corriendo iOS, más de un millón de aplicaciones disponibles y más de 60 mil millones de descargas acumuladas. Estos dispositivos se utilizan tanto en el ámbito personal como empresarial, almacenando información sensible que los convierte en objetivos de alto valor.

Un factor determinante en la superficie de ataque es la posibilidad de realizar *jailbreak* al dispositivo, lo que permite ejecutar aplicaciones no verificadas por Apple y otorga privilegios de usuario *root*. Aunado a esto, se ha contabilizado más de un millón de variantes de *malware* para dispositivos móviles, lo que justifica el estudio sistemático de la seguridad de las aplicaciones iOS.

2.2. Tipos de aplicaciones en iOS

Las aplicaciones iOS se clasifican en dos categorías principales según su arquitectura:

- **Aplicaciones nativas:** Desarrolladas en Objective-C con el framework Cocoa Touch. Son vulnerables principalmente a ataques de tipo *buffer overflow*.
- **Aplicaciones híbridas:** Implementan parte o la totalidad de su lógica en HTML, CSS y JavaScript. Están expuestas a ataques de *Cross Site Scripting* (XSS), *Cross Site Request Forgery* (XSRF) y también *buffer overflow*.

2.3. Clasificación de las pruebas en el contexto móvil

El artículo retoma la clasificación estándar de las pruebas de penetración aplicada al entorno iOS:

- **Caja Blanca:** El evaluador cuenta con código fuente y documentación. Permite detectar fallas lógicas, caminos incorrectos en el código, ciclos, condiciones y hoyos de seguridad interna.

- **Caja Negra:** Sin acceso al código, solo a la versión pública de la aplicación. Simula un ataque externo mediante intercepción de tráfico e inyección de contenido malicioso (XSS, inyección SQL, inyección de comandos).
- **Caja Gris:** Combinación de ambas, donde el código y la documentación actúan como guía de análisis.

El artículo destaca que un atacante solo necesita encontrar **un** vector de ataque exitoso, mientras que el evaluador profesional debe detectar la **mayor cantidad posible** de vulnerabilidades.

2.4. Vectores de ataque

La arquitectura de una aplicación móvil define tres vectores de ataque principales:

1. **Software en el dispositivo:** La aplicación en sí misma, sus datos locales y permisos.
2. **Canal de comunicación:** El tráfico de red entre la app y los servicios de backend.
3. **Infraestructura del servidor:** Los servicios y APIs que soportan la aplicación.

2.5. Metodología general

El proceso de prueba inicia con la fase de **information gathering** (recolección de información), cuyo objetivo es comprender el comportamiento esperado de la aplicación y las tecnologías que utiliza.

Los pasos metodológicos descritos son:

1. Familiarizarse con la aplicación como usuario común, navegando y entendiendo el flujo normal.
2. Configurar el dispositivo para utilizar un *proxy* (p. ej. **Burp Suite**) e interceptar el tráfico HTTP/HTTPS.
3. Analizar, modificar y reenviar paquetes para observar el comportamiento de la aplicación ante entradas manipuladas.
4. Complementar con herramientas de *sniffing* como **tcpdump** y **Wireshark** para ampliar el análisis de tráfico.
5. Prestar atención a: mensajes de error, inyección SQL, manejo de sesiones, criptografía débil, notificaciones Push, autenticación, autorización, almacenamiento local y validación de datos.

A partir del análisis del tráfico se pueden generar *fuzzing cases* o modelos de amenazas (como **STRIDE**) para identificar áreas de riesgo. El artículo también menciona el análisis en tiempo de ejecución, el estudio del almacenamiento local (logs, respaldos, SQLite, caché del teclado) y las implicaciones del contexto BYOD (*Bring Your Own Device*).

2.6. OWASP y checklist de evaluación

El *Open Web Application Security Project* (OWASP) proporciona una guía específica para la evaluación de aplicaciones iOS, que contempla dos checklists:

- **Checklist de plataforma:** Aspectos del sistema operativo iOS relevantes para la seguridad.
- **Checklist de red:** Aspectos relacionados con ataques de red y comunicaciones.

Estas listas sirven como marco de referencia para asegurar una cobertura completa durante la evaluación.

2.7. Amenazas emergentes

El artículo concluye señalando el incremento sostenido de aplicaciones maliciosas y amenazas móviles, incluyendo *malware*, gusanos, *ransomware*, *spam* SMS, *smishing* y botnets móviles. Recomienda que los usuarios estén conscientes de los privilegios que otorgan a las aplicaciones, el tipo de información que almacenan y si utilizan cifrado tanto para almacenamiento como para comunicación en red.

2.7.1. Referencia bibliográfica

Brito Abundis, C. J. (s.f.). Introducción a las pruebas de penetración en dispositivos iOS. *SG Buzz, Revista 42*, sección Prácticas. Instituto Politécnico Nacional / Software Guru. Recuperado de https://sg.com.mx/revista/42/introduccion-las-pruebas-penetracion--dispositivos-ios

3. Organizador Gráfico

El siguiente mapa mental sintetiza los temas principales del artículo *«Introducción a las pruebas de penetración en dispositivos iOS»* de Carlos Joaquín Brito Abundis (SG 42).

Figura 1: Mapa mental: Introducción a las pruebas de penetración en dispositivos iOS (Brito Abundis, SG 42)